UNIVERSIDADE VILA VELHA
ESPÍRITO SANTO

Aula 15 – Matrizes em C: Aplicações práticas

1. Objetivos de aprendizagem

Desenvolver programas em C que utilizem matrizes para armazenar, consultar e processar informações organizadas em linhas e colunas.

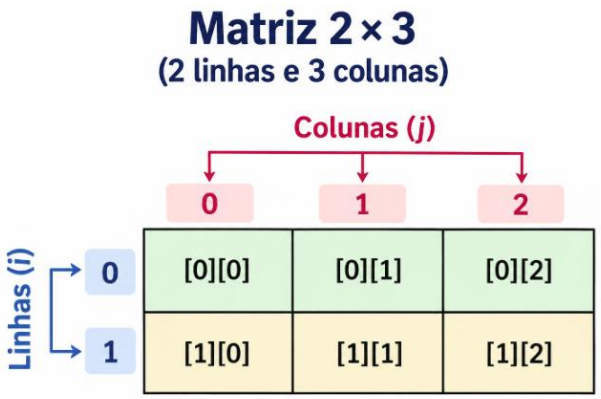
2. Relembrando... O que aprendemos sobre matrizes?

1. Qual é a diferença entre vetor e matriz?
2. Em int numeros[3][4], quantos elementos podem ser armazenados?
3. Por que utilizamos dois comandos for para percorrer uma matriz?

Análise o código abaixo e responda:

```
int numeros[2][3] = {
    {10, 20, 30},
    {40, 50, 60}
};
```

Qual valor está em numeros[1][2]?

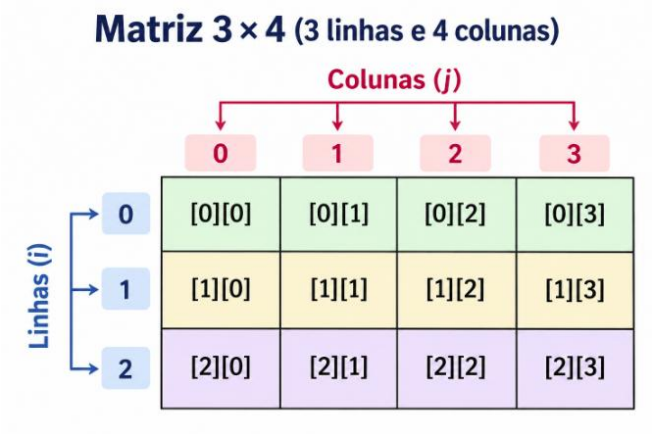


3. Situação-problema:

Uma loja precisa registrar a quantidade vendida de três produtos durante quatro dias.

Produto	Segunda	Terça	Quarta	Quinta
Camisetas	5	8	6	4
Calças	3	2	7	5
Bonés	4	6	3	8

Como descobrir a quantidade total de produtos vendidos? Como calcular o total vendido de cada produto?
A matriz permite organizar os dados e realizar esses cálculos sem precisar criar uma variável para cada venda.



Exemplo1: Somando todos os elementos

```
#include <stdio.h>

int main(void) {

    int vendas[3][4] = {

        {5, 8, 6, 4},

        {3, 2, 7, 5},

{4, 6, 3, 8}
```

```
    };

    int total = 0;

    for (int i = 0; i < 3; i++) {

        for (int j = 0; j < 4; j++) {

            total += vendas[i][j];

        }

    }

    printf("Total vendido: %d\n", total);

    return 0;

}
```

Total é uma variável acumuladora e que total += vendas[i][j] soma cada elemento da matriz ao total anterior.

Exemplo 2: Somando os valores de cada linha

Agora queremos descobrir quantas unidades de cada produto foram vendidas.

```
#include <stdio.h>

int main(void) {
    int vendas[3][4] = {
        {5, 8, 6, 4},
        {3, 2, 7, 5},
        {4, 6, 3, 8}
    };

    for (int i = 0; i < 3; i++) {
        int totalProduto = 0;

        for (int j = 0; j < 4; j++) {
            totalProduto += vendas[i][j];
        }

        printf("Produto %d: %d unidades\n",
               i + 1, totalProduto);
    }

    return 0;
}
```

Para realizar a soma das colunas de uma matriz em C, precisamos percorrer os elementos de cada coluna e acumular seus valores.

A principal diferença em relação à soma das linhas é que o for externo controla as colunas (j) e o for interno percorre as linhas (i).

Exemplo 3: Soma das colunas

Considere a seguinte matriz:

	Coluna 0	Coluna 1	Coluna 2
Linha 0	10	20	30
Linha 1	40	50	60
Soma	50	70	90

```
#include <stdio.h>

int main(void) {
    int matriz[2][3] = {
        {10, 20, 30},
        {40, 50, 60}
    };

    int soma;

    // Percorre as colunas
    for (int j = 0; j < 3; j++) {
        soma = 0;

        // Percorre as linhas
        for (int i = 0; i < 2; i++) {
            soma += matriz[i][j];
        }

        printf("Soma da coluna %d: %d\n", j, soma);
    }

    return 0;
}
```

Observação: O operador += permite somar um valor ao conteúdo atual de uma variável. Portanto, total += vendas[i][j] equivale a total = total + vendas[i][j].

Pergunta: Por que a variável totalProduto é inicializada com zero dentro do primeiro for?

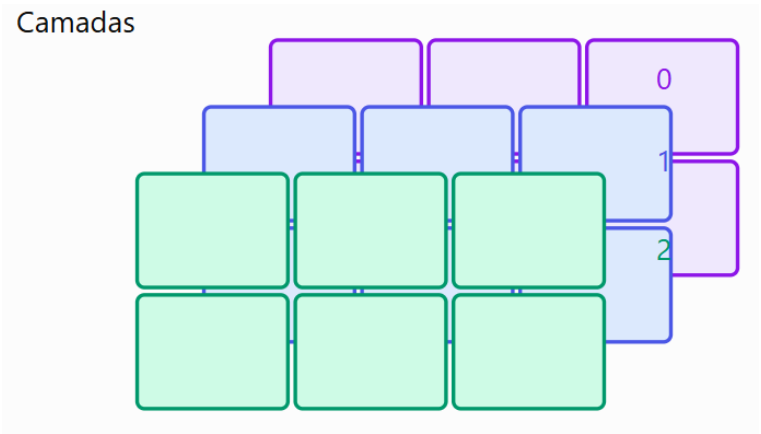
Curiosidade: embora a letra i seja frequentemente associada à palavra inglesa index (índice), o uso de i e j também tem raízes na notação matemática e nas primeiras linguagens de programação, como FORTRAN. J é a letra seguinte ao i no alfabeto e faz parte da sequência tradicional de índices i, j. Não é uma abreviação de uma palavra específica, assim como i não precisa necessariamente significar index.

4. É possível trabalhar com três dimensões em C?

A resposta é sim! A linguagem C permite trabalhar com arrays multidimensionais, incluindo estruturas tridimensionais.

O que é uma matriz tridimensional?

Uma matriz tridimensional é uma estrutura que armazena elementos do mesmo tipo, organizados em três dimensões. Podemos imaginá-la como várias matrizes bidimensionais empilhadas, formando camadas.



Como declarar em C?

int dados[2][3][4];

Exemplo do cotidiano

Imagine uma escola que precisa armazenar as notas de seus alunos em diferentes turmas. Uma matriz tridimensional poderia representar:

- Primeira dimensão: turmas.

- Segunda dimensão: alunos.
- Terceira dimensão: avaliações.

float notas[2][30][4];

Esse array pode armazenar quatro notas de cada um dos 30 alunos em duas turmas, totalizando 240 notas. Por exemplo:

notas[0][2][1] = 8.5;

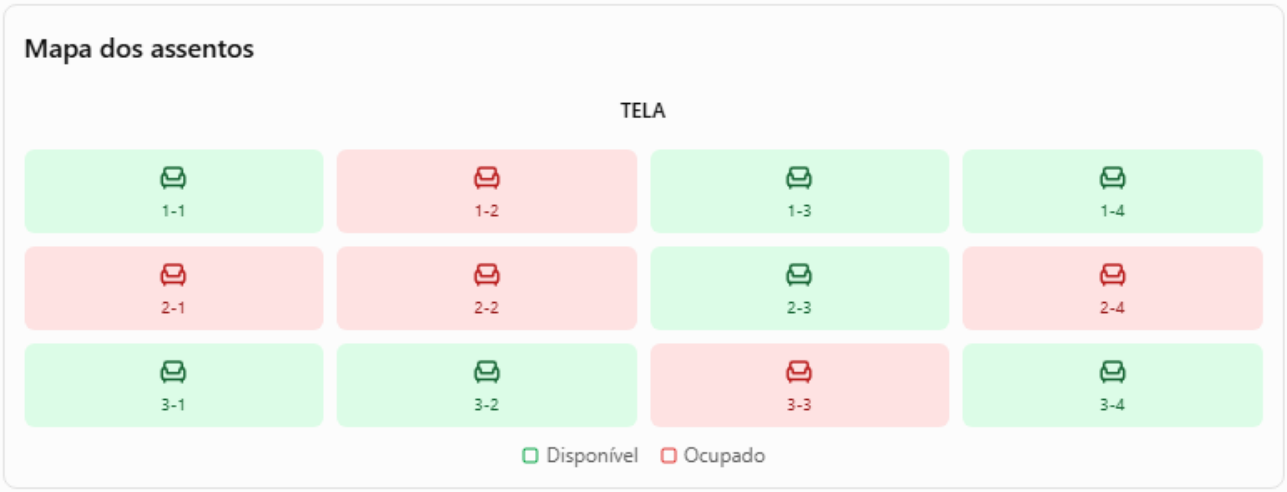
Uma estrutura tridimensional acrescenta uma terceira dimensão, que podemos imaginar como diferentes camadas de matrizes. Em C, podemos trabalhar com três ou mais dimensões, conforme a necessidade do problema.

EXERCÍCIO PARA ENTREGA:

Atividade – Reserva de assentos no cinema

Um cinema possui três fileiras com quatro assentos em cada uma. O sistema utiliza uma matriz de números inteiros para representar a ocupação dos assentos, sendo 0 para disponível e 1 para ocupado.

Desenvolva um programa em C que armazene a situação dos assentos em uma matriz 3×4, percorra todos os seus elementos utilizando estruturas de repetição e apresente a quantidade de assentos ocupados e disponíveis.



Exercícios de fixação

1. Ler os valores de uma matriz 2×3 e apresentar a soma de todos os elementos.

2. Armazenar as notas de três alunos em duas provas e calcular a média de cada aluno.

3. Criar uma matriz 3×3 e contar quantos valores são pares.

4. Criar uma matriz 3×3 e identificar o maior valor armazenado.

Questão de revisão:

Considere o código:

```
int numeros[2][2] = {
   {2, 4},
   {6, 8}
};

int soma = 0;

for (int i = 0; i < 2; i++) {
   for (int j = 0; j < 2; j++) {
      soma += numeros[i][j];
   }
}

printf("%d", soma);
```

Qual valor será exibido pelo programa?